Operating Systems

Youjip Won

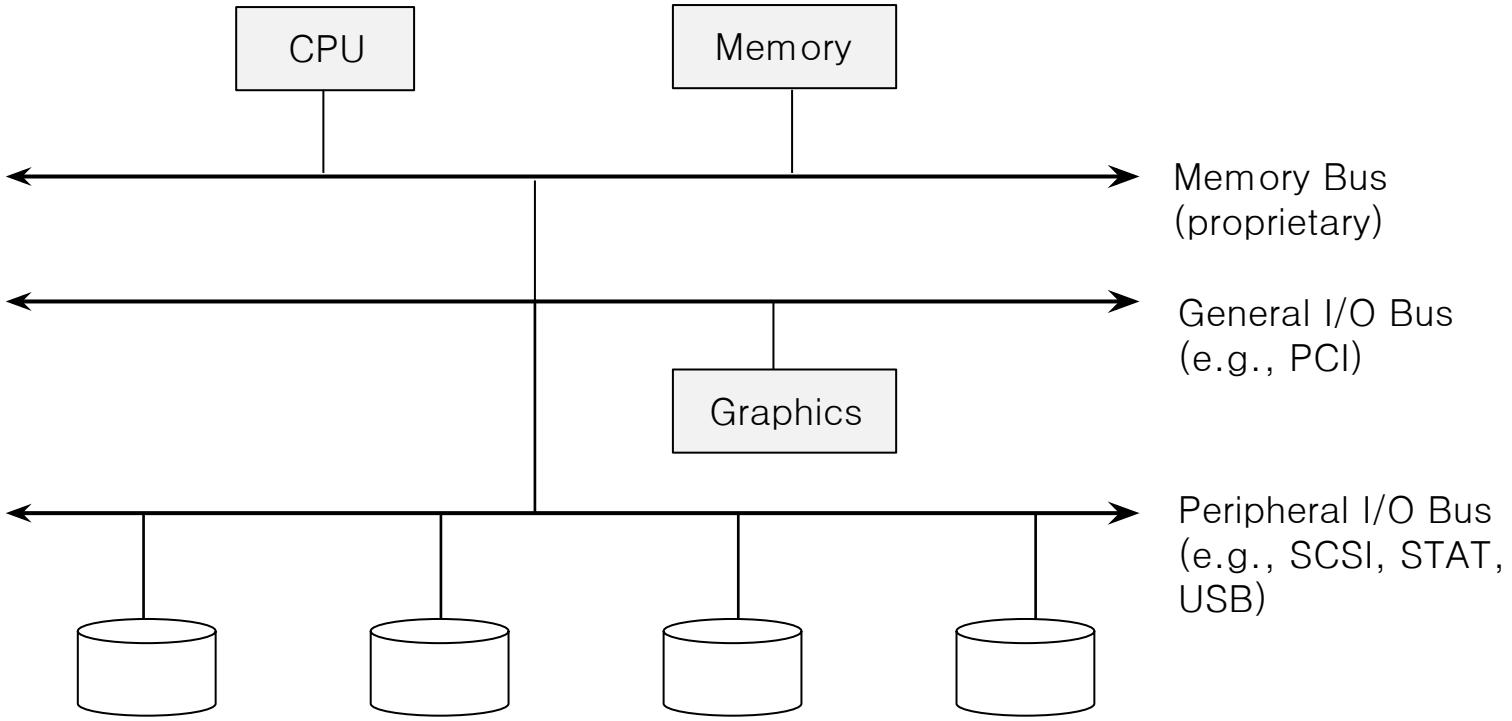
KAIST

36. I/O Devices

I/O Devices

- I/O is **critical** to computer system to **interact with systems.**
- Issue :
  - How should I/O be integrated into systems?
  - What are the general mechanisms?
  - How can we make the efficiently?

Structure of input/output (I/O) device



Prototypical System Architecture

CPU is attached to the main memory of the system via some kind of memory bus.

Some devices are connected to the system via a general I/O bus.

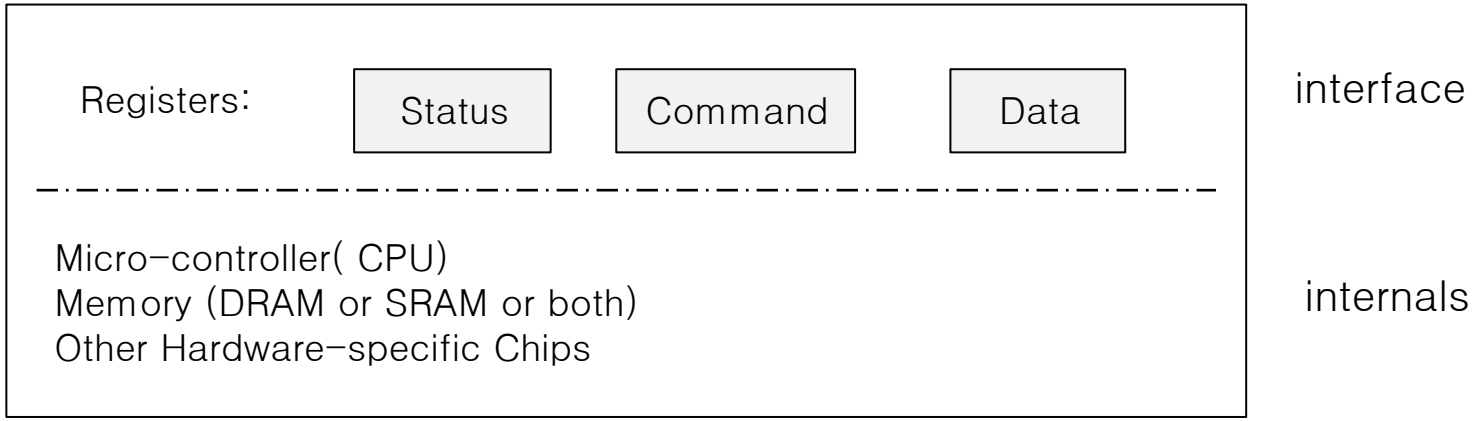
I/O Architecture

- Buses
  - Data paths that provided to enable information between CPU(s), RAM, and I/O devices.

- I/O bus
  - Data path that connects a CPU to an I/O device.
  - I/O bus is connected to I/O device by three hardware components: I/O ports, interfaces and device controllers.

Canonical Device

- Canonical Devices has two important components.
  - **Hardware interface** allows the system software to control its operation.
  - **Internals** which is implementation specific.



Canonical Device

Hardware interface of Canonical Device

- status register
  - See the current status of the device
- command register
  - Tell the device to perform a certain task
- data register
  - Pass data to the device, or get data from the device

By reading and writing above three registers ,
the operating system can control device behavior .

Hardware interface of Canonical Device (Cont.)

- Typical interaction example

```
while ( STATUS == BUSY)
    ; //wait until device is not busy
write data to data register
write command to command register
    Doing so starts the device and executes the command
while ( STATUS == BUSY)
    ; //wait until device is done with your request
```

Polling

- Operating system waits until the device is ready by **repeatedly** reading the status register.
  - Positive aspect is simple and working.
  - **However, it wastes CPU time just waiting for the device .**
    - Switching to another ready process is better utilizing the CPU.

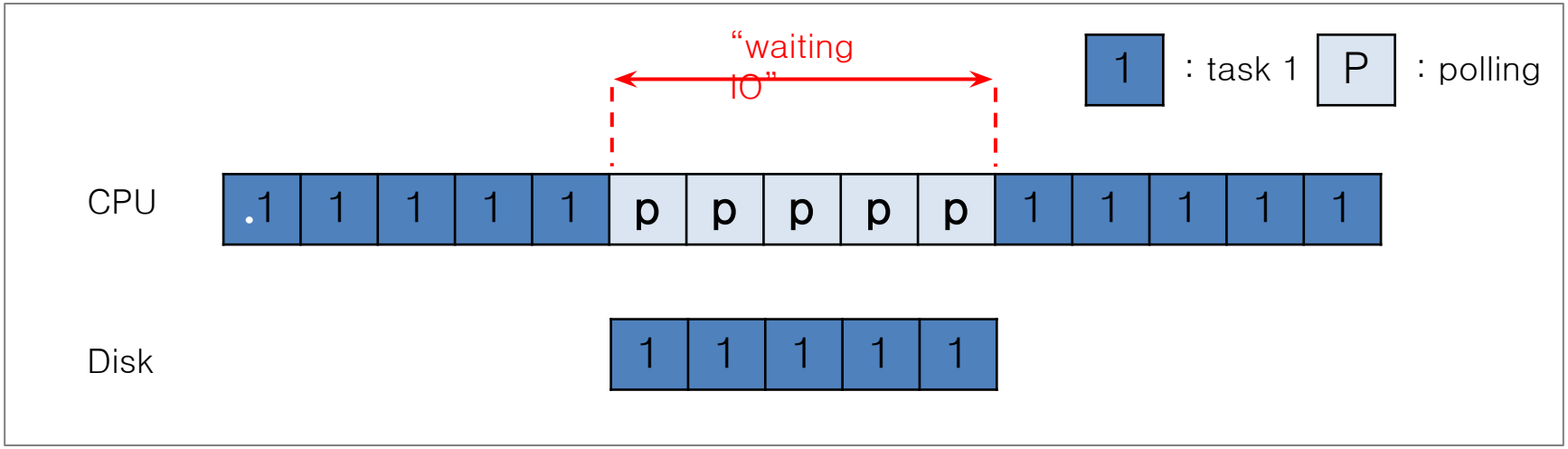


Diagram of CPU utilization by polling

interrupts

- **Put the I/O request process to sleep** and context switch to another.
- When the device is finished, wake the process waiting for the I/O by **interrupt** .
  - Positive aspect is allow to **CPU and the disk are properly utilized.**

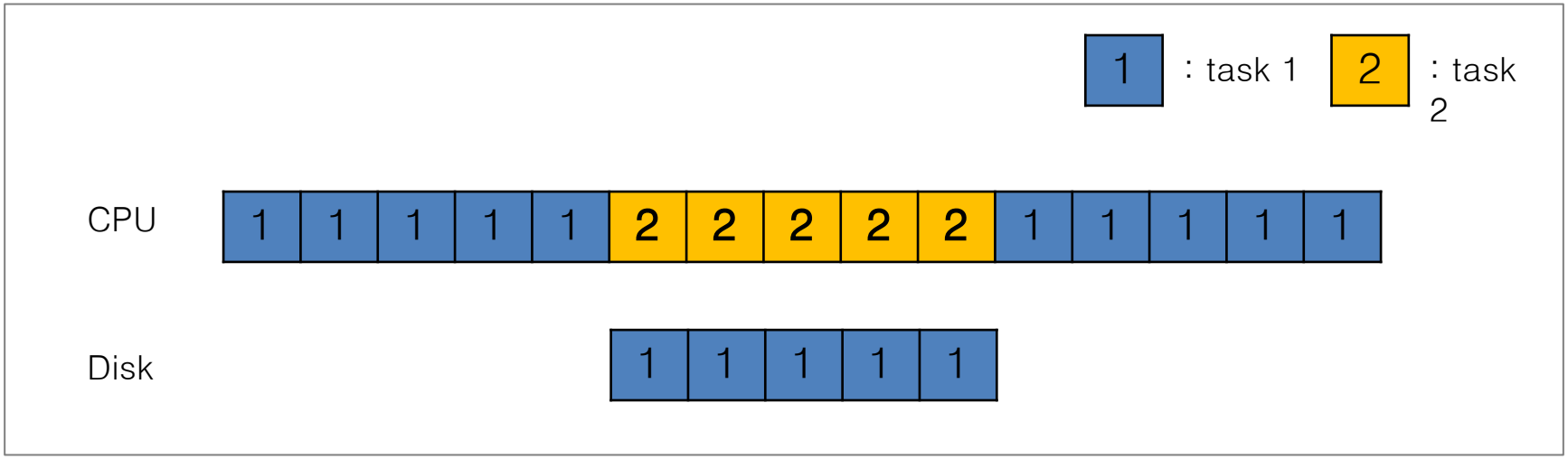


Diagram of CPU utilization by interrupt

Polling vs interrupts

- *However,* **"interrupts is not always the best solution"**
  - If, device performs very quickly, interrupt will "slow down" the system.
  - Because **context switch is expensive (switching to another process)**

If a device is fast ☐ poll is best.
If it is slow ☐ interrupt s is better.

CPU is once again over-burdened

- CPU **wastes a lot of time** to copy the *a large chunk of data* from memory to the device.

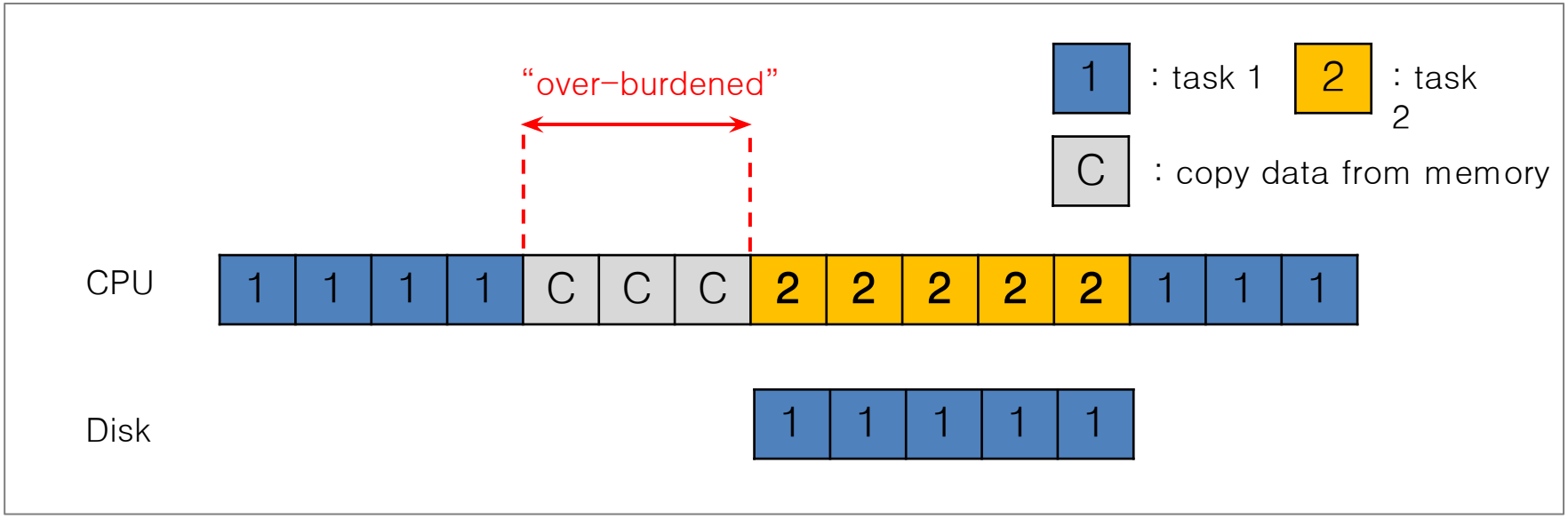


Diagram of CPU utilization

DMA (Direct Memory Access)

- **Copy data** in memory by knowing "where the data lives in memory, how much data to copy"
- When completed, DMA raises an interrupt, I/O begins on Disk.

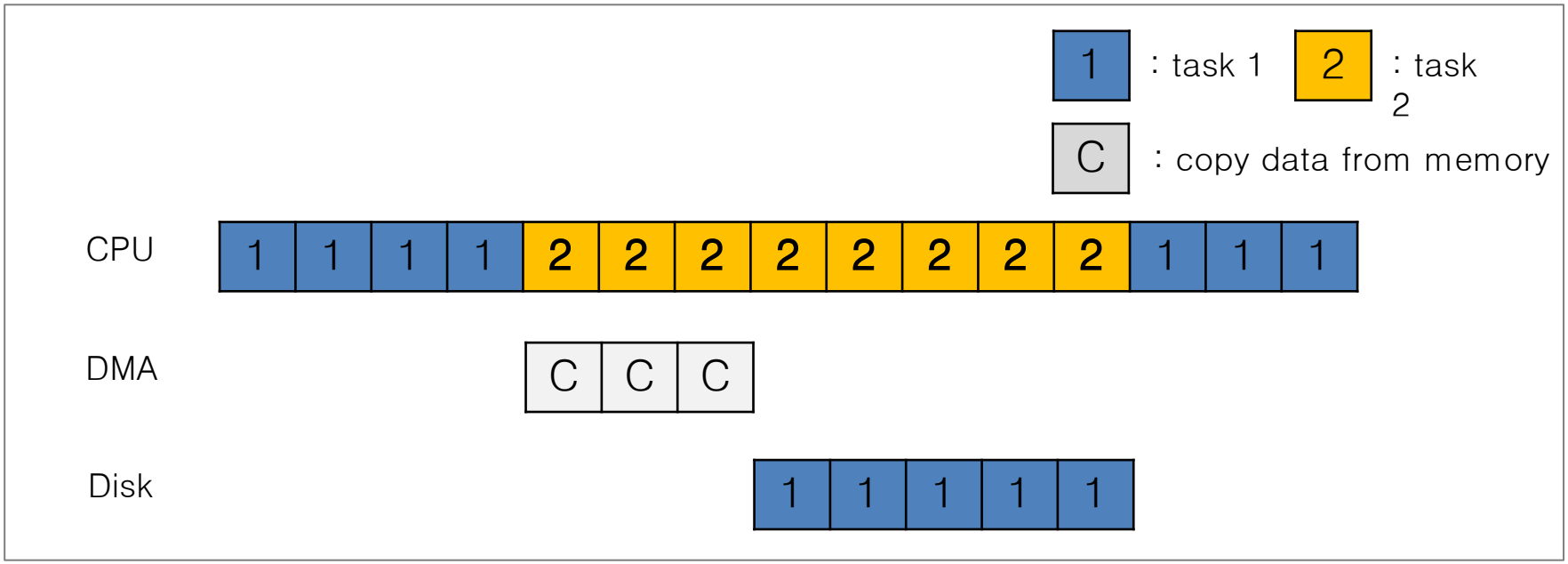


Diagram of CPU utilization by DMA

Device interaction

- How the OS communicates with the **device**?
- Solutions
  - I/O instructions: a way for the OS to send data to specific device registers.
    - Ex) `in` and `out` instructions on x86
  - memory-mapped I/O
    - Device registers available as if they were memory locations.
    - The OS `load` (to read) or `store` (to write) to the device instead of main memory.

IDE Device Driver

- For registers
  - Control register (8 bit)
  - Command block registers
  - Status register (8 bit)
  - Error register (8 bit)

The IDE Interface

- Control Register (1 Byte):

  Address 0x3F6 = 0x08 (0000 1RE0): R=reset, E="Enable Interrupt", E=0 means "enable interrupt"

- Command Block Registers:

  Address 0x1F0 = Data Port (128 Byte)

  Address 0x1F1 = Error (1 Byte)

  Address 0x1F2 = Sector Count (1 Byte)

  Address 0x1F3 = LBA low byte (1 Byte)

  Address 0x1F4 = LBA mid byte (1 Byte)

  Address 0x1F5 = LBA hi byte (1 Byte)

  Address 0x1F6 = 1B1D TOP4LBA: B=LBA, D=drive

  Address 0x1F7 = Command/status(1 Byte)

- Status Register (Address 0x1F7): (1 Byte)

  7 6 5 4 3 2 1 0

  BUSY READY FAULT SEEK DRQ CORR IDDEX ERROR

The IDE Interface (Cont.)

- Error Register (Address 0x1F1): (check when Status ERROR==1)

7	6	5	4	3	2	1	0
BBK	UNC	MC	IDNF	MCR	ABRT	T0NF	AMNF

BBK = Bad Block,

UNC = Uncorrectable data error,

MC = Media Changed,

IDNF = ID mark Not Found,

MCR = Media Change Requested,

ABRT = Command aborted,

T0NF = Track 0 Not Found,

AMNF = Address Mark Not Found

The basic protocol of IDE device driver

- Wait for the drive to be ready.
- Write parameters to command registers.
  - Sector count
  - logical block address (LBA) of sector
  - Driver number
- Issue read/write to command register.
  - Read or Write
  - For write command, transfer data
- Handle interrupt.
- Handle Error.

IDE device driver in xv6

```
void ide_rw(struct buf *b) {
    acquire(&ide_lock);
    for (struct buf **pp = &ide_queue; *pp; pp=&(*pp)->qnext) ;
    *pp = b;                                                    1
    if (ide_queue == b)                 // if q is empty,
      ide_start_request(b);             // send request to disk.   2
    while ((b->flags & (B_VALID|B_DIRTY)) != B_VALID)
      sleep(b, &ide_lock);              // wait for completion     3
    release(&ide_lock);
}
```

Mechanism: add the buffer to the `ide_queue` and perform IO.

1. Enqueu the buffer to the `ide_queue`.
2. If the buffer is the only entry in the `ide_queue`, start the IO right away.
3. Wait for the completion of the IO.
    1. If the IO is READ, interrupt handler resets the `B_VALID` flag.
    2. If the IO is write, interrupt handler resets the `B_DIRTY` flag.

Device driver

```
static void ide_start_request(struct buf *b) {
    ide_wait_ready();
    outb(0x3f6, 0); // generate interrupt
    outb(0x1f2, 1); // the count of sectors
    outb(0x1f3, b->sector & 0xff);
    outb(0x1f4, (b->sector >> 8) & 0xff);
    outb(0x1f5, (b->sector >> 16) & 0xff);
    outb(0x1f6, 0xe0 | ((b->dev&1)<<4) | ((b->sector>>24)&0x0f));
    if (b->flags & B_DIRTY){
            outb(0x1f7, IDE_CMD_WRITE); // WRITE
            outsl(0x1f0, b->data, 512/4); // transfer data
    } else {
        outb(0x1f7, IDE_CMD_READ); // READ
    }
}
```

Device driver

```
static void ide_start_request(struct buf *b) {
    ide_wait_ready();
    ...
}
```

Call

```
static int ide_wait_ready() {
    int r;
    while (((r = inb(0x1f7)) & IDE_BSY) || !(r & IDE_DRDY))
        ;
}
```

1. Read Status Register(0x1F7) and save it to r.
2. Check if the drive is the busy. (IDE_BSY).
3. Check if the drive is ready .
4. If the device is not busy and ready, get out of the while loop.

Status Register is also used as a command register. The device uses it to store the status of the register. The host uses it to store the command.

Enable interrupt and set the sector count.

```
static void ide_start_request(struct buf *b) {
    ide_wait_ready();
    outb(0x3f6, 0); // generate interrupt
    outb(0x1f2, 1); // the number of sectors
    ...
```

1. Enable interrupt for the device: set the Control Register(0x3F6).
2. Set the number of sectors for an IO: set the sector count register(0x1F2).

Specifying the LBA

```
static void ide_start_request(struct buf *b) {
    ide_wait_ready();
    outb(0x3f6, 0); // generate interrupt
    outb(0x1f2, 1); // the count of sectors
    outb(0x1f3, b->sector & 0xff);          // 1
    outb(0x1f4, (b->sector >> 8) & 0xff);   // 2
    outb(0x1f5, (b->sector >> 16) & 0xff); // 3
    outb(0x1f6, 0xe0 | ((b->dev&1)<<4) | ((b->sector>>24)&0x0f)); // 4
```

Write Command and the data transfer

```
static void ide_start_request(struct buf *b) {
    ...
    outb(0x1f3, b->sector & 0xff);
    outb(0x1f4, (b->sector >> 8) & 0xff);
    outb(0x1f5, (b->sector >> 16) & 0xff);
    outb(0x1f6, 0xe0 | ((b->dev&1)<<4) |
    ((b->sector>>24)&0x0f));
    if (b->flags & B_DIRTY){
            outb(0x1f7, IDE_CMD_WRITE); // WRITE
            outsl(0x1f0, b->data, 512/4); // transfer data
    }
```

1. If the DIRTY bit is set, perform write. (In xv6, device driver performs write only when the buffer is dirty.)
2. Set the write command at the Command Register (0x1F7).
3. Write the data to the Data Register (0x1F0).
    1. The amout of data to write: 512/4 byte
    2. `static inline void outsl (int port, const void *addr, int cnt)`

Read command

```
static void ide_start_request(struct buf *b) {
    ...
    outb(0x1f6, 0xe0 | ((b->dev&1)<<4) | ((b->sector>>24)&0x0f));
    if (b->flags & B_DIRTY){
            outb(0x1f7, IDE_CMD_WRITE); // WRITE
            outsl(0x1f0, b->data, 512/4); // transfer data
    } else {
        outb(0x1f7, IDE_CMD_READ); // READ
    }
}
```

1. If the buffer is not dirty, perform read.
2. Set the command READ at Command Register (0x1F7).
3. Perform read.

Interrupt handler (IO Completion)

```
void ide_intr() {
    struct buf *b;
    acquire(&ide_lock);
    if (!(b->flags & B_DIRTY) && ide_wait_ready() >= 0)
        insl(0x1f0, b->data, 512/4); // if READ: get data
    b->flags |= B_VALID;
    b->flags &= B_DIRTY;
    wakeup(b); // wake waiting process
    if ((ide_queue = b->qnext) != 0)   // start next request
        ide_start_request(ide_queue);  // (if one exists)
    release(&ide_lock);
}
```

Read

Set the flag properly.
→wake up from `ide_rw()`

Start the next request in the `ide_queue`.

Device interaction (Cont.)

- How the OS interact with **different types of interfaces** ?
  - Ex) We'd like to build a file system that worked on top of SCSI disks, IDE disks, USB keychain drivers, and so on.
- Solutions: Abstraction
  - Abstraction encapsulate any specifics of device interaction.

Summary

- To save the CPU cycles for IO
  - Use Interrupt
  - Use DMA
- To access the device registers
  - Memory mapped IO
  - Explicit IO instruction